

Scaling, Train-Test Split & Pipelines

Contents

1	Feature Scaling	2
1.1	Why is scaling needed?	2
1.2	Min-Max Scaling	2
1.2.1	Concept	2
1.2.2	Formula	2
1.2.3	Worked Example	2
1.2.4	Python	3
1.3	Standard Scaling (Z-score)	3
1.3.1	Concept	3
1.3.2	Formula	3
1.3.3	Worked Example	3
1.3.4	Python	4
1.4	Robust Scaling	4
1.4.1	Concept	4
1.4.2	Formula	4
1.4.3	Worked Example (the point of robust scaling)	4
1.4.4	Python	5
1.5	Comparison: which one to use?	5
2	Train-Test-Validation Split	6
2.1	The three sets	6
2.2	Typical splits	6
2.3	Python	6
3	Data Leakage	7
3.1	Concept	7
3.2	A common way leakage sneaks in	7
3.3	Correct Python	7
4	sklearn Pipeline	8
4.1	Concept	8
4.2	Why pipelines are worth it	8
4.3	The workflow inside a pipeline	8
4.4	Python	8

1 Feature Scaling

Sometimes one column (feature) has very large numbers while another column has very small numbers. Look at this small table of students:

Student	Age	Annual Income (Rs.)
A	20	250000
B	22	800000
C	21	500000

Here **Age** ranges only from 20 to 22, but **Income** ranges from 250000 to 800000. The income numbers are thousands of times larger than the age numbers.

Key Idea

Feature scaling changes the range of numerical features so that they sit on a *similar scale*. After scaling, no single column is allowed to dominate just because its raw numbers happen to be bigger.

1.1 Why is scaling needed?

Without scaling:

- Large-valued features dominate small-valued features.
- Different units (years, kilograms, rupees, ...) make values hard to compare.
- Many numerical calculations become less stable.

Scaling gives every feature a comparable numerical range, so each one gets a fair say.

1.2 Min–Max Scaling

1.2.1 Concept

Min–Max scaling squeezes every value into a fixed range, usually **0 to 1**.

- The smallest value becomes **0**.
- The largest value becomes **1**.
- Every other value lands somewhere in between.

1.2.2 Formula

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

where x is the original value, $x_{\min}$ is the smallest value in the column, and $x_{\max}$ is the largest value in the column.

1.2.3 Worked Example

Take the marks: 40, 60, 80, 100. Here $x_{\min} = 40$ and $x_{\max} = 100$, so the range is $100 - 40 = 60$.

For 80:

$$x' = \frac{80 - 40}{100 - 40} = \frac{40}{60} = 0.667$$

Doing this for every value gives:

Marks	Scaled (0–1)
40	0.000
60	0.333
80	0.667
100	1.000

1.2.4 Python

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

df = pd.DataFrame({"Marks": [40, 60, 80, 100]})

scaler = MinMaxScaler()
df["Scaled"] = scaler.fit_transform(df[["Marks"]])

print(df)
```

1.3 Standard Scaling (Z-score)

1.3.1 Concept

Standard scaling re-centres the data so that:

- the **mean** becomes **0**, and
- the **standard deviation** becomes **1**.

The scaled value tells us *how many standard deviations a value is away from the average*. A positive result means above average; a negative result means below average.

1.3.2 Formula

$$z = \frac{x - \mu}{\sigma}$$

where μ is the mean of the column and σ is its standard deviation.

1.3.3 Worked Example

Marks: 40, 60, 80, 100.

Step 1 — Mean.

$$\mu = \frac{40 + 60 + 80 + 100}{4} = \frac{280}{4} = 70$$

Step 2 — Standard deviation.

$$\sigma = \sqrt{\frac{(40 - 70)^2 + (60 - 70)^2 + (80 - 70)^2 + (100 - 70)^2}{4}} = \sqrt{\frac{2000}{4}} = \sqrt{500} \approx 22.36$$

Step 3 — Z-score for each value. For 80:

$$z = \frac{80 - 70}{22.36} = \frac{10}{22.36} \approx 0.447$$

Marks	Standardized
40	-1.342
60	-0.447
80	0.447
100	1.342

Note

Small detail. `StandardScaler` divides by N (the count), not by $N - 1$. This is called the *population* standard deviation. If you compute the standard deviation in a different tool that uses $N - 1$, your hand answer may differ slightly from the code output. The idea stays the same.

1.3.4 Python

```
from sklearn.preprocessing import StandardScaler
import pandas as pd

df = pd.DataFrame({"Marks": [40, 60, 80, 100]})

scaler = StandardScaler()
df["Standardized"] = scaler.fit_transform(df[["Marks"]])

print(df)
```

1.4 Robust Scaling

1.4.1 Concept

Robust scaling is built to survive **outliers** — a few values that are far larger or far smaller than the rest. Instead of mean and standard deviation, it uses:

- the **median** (the middle value), and
- the **interquartile range (IQR)**.

Because the median and IQR ignore the extreme ends of the data, a single huge value does not throw the whole scaling off.

Quick reminder. Sort the data. The **median** is the middle value. Q_1 (the first quartile) is the value with one quarter of the data below it, and Q_3 (the third quartile) is the value with three quarters below it. The middle 50% of the data lies between Q_1 and Q_3 .

1.4.2 Formula

$$x' = \frac{x - \text{Median}}{IQR}, \quad \text{where } IQR = Q_3 - Q_1$$

1.4.3 Worked Example (the point of robust scaling)

Marks: 40, 45, 50, 55, **500**. The value **500** is a clear outlier.

Here median = 50, $Q_1 = 45$, $Q_3 = 55$, so $IQR = 55 - 45 = 10$. For 40: $x' = \frac{40 - 50}{10} = -1.0$.

Now compare what *Min-Max* and *Robust* scaling do to the same data:

Marks	Min-Max	Robust
40	0.000	-1.0
45	0.011	-0.5
50	0.022	0.0
55	0.033	0.5
500	1.000	45.0

Why this matters

With **Min-Max**, the outlier 500 stretches the range so much that the four normal marks (40, 45, 50, 55) are crushed into almost the same tiny numbers (0, 0.011, 0.022, 0.033) — we can barely tell them apart anymore. With **Robust** scaling, those same four marks stay nicely spread out (−1, −0.5, 0, 0.5), and the outlier simply shows up as a large 45. The useful information is preserved.

1.4.4 Python

```
from sklearn.preprocessing import RobustScaler
import pandas as pd

df = pd.DataFrame({"Marks": [40, 45, 50, 55, 500]})

scaler = RobustScaler()
df["Robust"] = scaler.fit_transform(df[["Marks"]])

print(df)
```

1.5 Comparison: which one to use?

Method	What it does	Hurt by outliers?	Use when...
Min-Max	Fits values into a fixed range 0 to 1	Yes	you need a fixed bounded range and the data has no big outliers
Standard	Mean = 0, Std = 1	Yes	the data is roughly evenly spread around its average
Robust	Centres on median, divides by IQR	No	the data contains outliers you do not want to remove

2 Train–Test–Validation Split

Suppose you have 1000 student records. It is a mistake to use all of them at once. Instead, we divide the records into separate groups, each with its own job.

The exam analogy

Think about how a student prepares for an exam:

- **Training set** = the *textbook problems* you study and learn from.
- **Validation set** = the *mock tests* you take to check yourself and adjust how you study.
- **Test set** = the *final exam*, which you see only once, to measure how well you really learned.

If you had already seen the final exam paper while studying, your score would not be honest. That is exactly why the test set is kept hidden until the very end.

2.1 The three sets

Training set Used to *learn* the patterns in the data. (e.g. 700 records)

Validation set Used to *compare* different settings and pick the better one. (e.g. 150 records)

Test set Used *only once at the very end* to measure final performance. (e.g. 150 records)

2.2 Typical splits

Three-way split		Two-way split	
Training	70%	Training	80%
Validation	15%	Test	20%
Test	15%		

2.3 Python

```
from sklearn.model_selection import train_test_split
import pandas as pd

df = pd.DataFrame({
    "Marks": [45, 55, 65, 75, 85],
    "Result": [0, 0, 1, 1, 1]
})

X = df[["Marks"]] # input feature(s)
y = df["Result"]  # the answer we want to predict

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)
```

Note

`random_state=42` just fixes the shuffling so that everyone running the code gets the *same* split. Any fixed number works; the point is repeatability.

3 Data Leakage

3.1 Concept

Data leakage happens when information from the test data accidentally sneaks into the preparation or training of the model. When this happens, the model effectively gets a peek at the answers it was supposed to be tested on. The results then look great during practice but fall apart on truly new data, because the good score was not honest.

3.2 A common way leakage sneaks in

A very common mistake is **scaling the entire dataset before splitting it**.

Incorrect order (leakage)	Correct order (no leakage)
1. Scale the <i>entire</i> dataset	1. Split the data first
2. Split into train and test	2. Fit the scaler on <i>training</i> data
	3. Transform the training data
	4. Transform the test data using that <i>same</i> scaler

In the incorrect order, the scaler computed its minimum, maximum, mean, etc. using the test rows too. So the test data has already influenced the preparation — the test set is no longer truly unseen.

The golden rule

Fit only on the training data. The scaler (or any preprocessing step) must *learn* its numbers from the training set alone, and then simply *apply* those same numbers to the test set.

3.3 Correct Python

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)    # LEARN + apply on
train                                     # only APPLY on test
X_test  = scaler.transform(X_test)
```

Remember the difference:

`fit_transform()` — used *only* on the training data. It learns the numbers (min, max, mean, ...) and applies them.

`transform()` — used on the test data. It only applies the numbers already learned from training. It never looks at the test data to learn anything new.

4 sklearn Pipeline

4.1 Concept

Preprocessing usually needs several steps in a fixed order: handle missing values, encode text columns, then scale the numbers. Writing each step by hand is repetitive and easy to get wrong. A **Pipeline** bundles all these steps into one neat object that runs them in the correct order automatically.

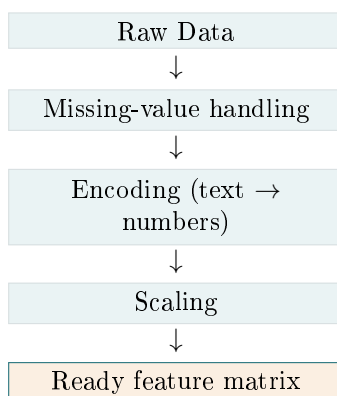
Key Idea

A **Pipeline** chains multiple steps together so they always run in the same order. You call `fit` once, and every step happens for you — in sequence, every time.

4.2 Why pipelines are worth it

- **Order is guaranteed.** Steps cannot accidentally run out of order.
- **Less code, fewer mistakes.** One object instead of many loose lines.
- **It prevents data leakage automatically.** When you call `fit` on the pipeline with the training data, every step is fitted on training data only; on the test data, every step only transforms. The golden rule from the previous section is enforced for you.

4.3 The workflow inside a pipeline



4.4 Python

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipeline.fit(X_train, y_train)          # runs scaler, then model
predictions = pipeline.predict(X_test)
```

Note

Each step is written as a pair: a short *name* you choose (like `"scaler"`) and the *object* that does the work (like `StandardScaler()`). When real data has *both* text and number columns that need different treatment, a **Pipeline** is usually combined with a **ColumnTransformer** so each column type gets the right step. You will see a complete example of this in the companion teaching notebook.